RECOMMENDED TECH STACK

AI-Powered Productivity App

Android + iOS | Version 1.0 | Reviewed 9 October 2026

DECISION
React Native + Expo + TypeScript, Supabase/PostgreSQL, focused Swift/Kotlin native modules, and a server-side AI Action Engine with a replaceable model provider.

1. Executive decision

Use React Native + Expo + TypeScript for most shared application code. Keep OS-sensitive behavior behind native service interfaces, especially scheduled alarms and app blocking. Use Supabase as the initial backend and make AI actions typed, validated and server-controlled. This balances fast product iteration with the native integration needed by this app.

This is a modular-monolith starting point, not a commitment to microservices. Structure the code so that native implementations and AI providers can change without rewriting feature screens.

2. Recommended stack at a glance

Layer	Recommendation	Role
Mobile app	React Native + Expo + TypeScript	Shared iOS/Android UI and product logic
Navigation	Expo Router	File-based routes, typed navigation and deep links
UI / icons	React Native StyleSheet + design tokens; lucide-react-native	Consistent design-system implementation
Animation	Reanimated + Gesture Handler	Smooth, restrained transitions and gestures
Client state	Zustand	Transient UI state
Server state	TanStack Query	Fetching, cache, retry and mutation state
Forms / validation	React Hook Form + Zod	Safe forms and runtime schemas
Local data	Expo SQLite	Local cache, drafts and pending action queue
Cloud backend	Supabase / PostgreSQL / Auth / Storage / Edge Functions	Accounts, sync, owned data and server actions
AI	Server-side provider adapter; start with OpenAI Responses API tools + Structured Outputs	Typed AI plans and controlled actions
Voice	Expo audio capture + server-side speech-to-text adapter	Consistent voice-to-command pipeline
Alarms	iOS AlarmKit where available; Android native AlarmManager	System-scheduled alarms
Focus blocking	iOS Screen Time frameworks; Android implementation subject to feasibility and policy	Platform-specific enforcement
Notifications	Expo Notifications with APNs/FCM for ordinary notifications	Reminders and updates; alarms handled natively
Subscriptions	RevenueCat	In-app purchase entitlement management
Analytics / crash	PostHog + Sentry	Product learning, crash and error tracking
Admin	Next.js + TypeScript + Supabase	Protected configuration and support tools
Build / release	EAS Build, EAS Submit, EAS Update, GitHub Actions	CI, internal builds and store releases
Testing	Jest, React Native Testing Library, Maestro + native-device tests	Unit, UI and end-to-end validation

3. Mobile architecture

- Use TypeScript throughout the mobile app and action orchestration layer.
- Use Expo Router for screens and deep links; keep reusable components outside route files.
- Use centralized design tokens for colors, typography, spacing, radii, surfaces and feature accents. Treat Stitch/Figma as design references; implement the actual mobile app in React Native rather than shipping exported HTML.
- Keep platform-specific capabilities behind interfaces such as AlarmService, FocusBlockingService, NotificationService and SpeechToTextProvider.

- Use Zustand only for transient client UI state. Use TanStack Query for remote data, and SQLite for local drafts/cache and a durable pending-action queue.
- Use Supabase/PostgreSQL as the cloud source of truth. Define sync conflict behavior and idempotent retries before broad offline editing.

4. Native capabilities that must be tested early

Alarms

- Android: a native Kotlin module using AlarmManager and the appropriate exact-alarm capability. Check permission at schedule time and explain how to recover if it is denied or revoked.
- iOS: evaluate AlarmKit for supported OS versions. Confirm the minimum iOS target and fallback experience before promising parity.
- Do not depend on JavaScript timers or a cloud notification as the alarm mechanism. Test while locked, after reboot, with app terminated, across time-zone changes, and under permission/battery restrictions.

Focus / app blocking

- iOS: evaluate FamilyControls, ManagedSettings and DeviceActivity. Distribution requires the Apple Family Controls entitlement for the app and applicable extensions.
- Android: run an early native proof of concept and policy review. Blocking behavior depends on chosen APIs, permissions and store restrictions; do not promise identical hard-blocking behavior on every device until verified.
- If platform limitations prevent a promised behavior, clearly explain the limitation and show a supported alternative instead of presenting a timer as an app blocker.

Expo workflow

Use Expo development builds and Continuous Native Generation when custom Swift/Kotlin modules are required. EAS Update can deliver compatible JavaScript, styling and assets; native code, permissions or native dependencies require a new build.

5. AI Action Engine

The central AI feature is not just a chat screen. It is a controlled action pipeline that turns voice or text into a validated list of operations across Alarm, Tasks, Calendar, Focus, Notes and supported Workspace integrations.

```
Voice/text → transcription → intent and action plan → schema validation → permission/conflict checks →
preview/confirmation → controlled executor → per-action result log
```

Required engineering rules

1. Keep model API keys and provider credentials on the server only.
2. Put AI behind an AIProvider interface. Start with one model provider; keep model configuration replaceable.
3. Use tool/function calling or structured outputs to return typed action objects. Validate each object with Zod on the server.
4. Use an allowlist of tools such as create_task, create_note, schedule_alarm, create_calendar_event, start_focus_session and search_connected_items.
5. Never allow the model to execute arbitrary SQL, shell commands or application code.
6. Ask a clarifying question when a time, date, target or intent is ambiguous. Require explicit confirmation for destructive, external, shared or consequential actions.
7. Use idempotency keys and command/action IDs to prevent duplicate tasks, alarms and events when requests retry.
8. Track each action status and report partial success and failures for multi-action requests.
9. Record audio only after an explicit user action. Delete raw audio by default after transcription unless the user opts into retention.
10. Add timeouts, rate limits, usage budgets, provider fallback and safe failure handling.

Voice pipeline

Use native-friendly audio capture, then send the recording through a speech-to-text provider adapter. Feed its transcript through exactly the same intent, validation and action pipeline as typed input. Keep speech-to-text, language-model planning and tool execution as separate modules.

6. Backend, data and integrations

Supabase

Use PostgreSQL for users, tasks, projects, events, notes, focus sessions, integrations, AI commands, action logs and subscription entitlements. Use Supabase Auth, Storage and Edge Functions as needed; enable Row Level Security (RLS) for user-owned data. Use Realtime only where a feature needs it.

- Keep database migrations in version control and separate development, staging and production.
- Never place a Supabase service-role key or third-party secret in the mobile application.
- Use schema validation, least-privilege access and audit logs for privileged actions.

Integration order

1. Google Calendar and Gmail, using minimal OAuth scopes and clear consent.
2. Outlook / Microsoft 365 Mail and Calendar.
3. Notion or Google Drive based on validated user demand.
4. Expand only after measuring actual usage, API limits and support burden.

OAuth callbacks, token refresh, webhook verification and sync jobs should run server-side. Encrypt provider tokens at rest, allow disconnect/revoke, and delete stored credentials on disconnect. Never claim access to content that a provider API does not expose.

Subscriptions, analytics and errors

- RevenueCat for subscription entitlements and in-app purchase integration; test using development builds.
- PostHog for privacy-reviewed product analytics. Do not send message bodies, raw audio, passwords, tokens or other sensitive content to analytics.
- Sentry for crash/error diagnostics. Remove personal content from error logs by default.

7. Admin panel and release pipeline

Build the admin panel as a separate Next.js + TypeScript application with role-based access. Include remote configuration, feature flags, managed content/news sources, subscription-plan settings, support diagnostics and safe announcement controls. Do not let the admin panel run arbitrary executable code on users' devices.

Use EAS Build for native binaries, EAS Submit for store submission, and EAS Update only for compatible JavaScript/assets/UI updates. Maintain development, preview/staging and production environments. Test permission and alarm changes with internal builds before release.

OFFICIAL DOCUMENTATION

Expo Router | Expo native modules | Supabase + Expo | Android alarms
Apple AlarmKit | Apple Family Controls | OpenAI function calling | Structured Outputs
RevenueCat + Expo | EAS Build | EAS Update

8. Suggested repository structure

```
app/
  src/app/                # Expo Router screens and layouts
  src/components/         # Reusable UI components
  src/design-system/      # Theme and component tokens
  src/features/           # alarm, tasks, calendar, focus, workspace, notes, news, ai-command
  src/services/           # Typed backend/native interfaces
  src/state/              # Zustand stores
  src/data/               # TanStack Query, SQLite and sync
  src/schemas/            # Zod schemas
  src/lib/                # Supabase client and utilities
  modules/                 # Custom Expo native modules
  supabase/migrations/     # Versioned database schema
  supabase/functions/      # Server-side functions
  admin/                   # Separate Next.js admin app
  tests/
  docs/
```

9. Required proof-of-concept gates

1. Alarm spike: schedule, fire, snooze and cancel on real iOS and Android devices while locked; test permission denial, reboot and timezone changes.
2. App-blocking spike: verify enforcement on supported OS versions, entitlements, user consent and store policies before committing to cross-platform promises.
3. AI spike: one voice command generates multiple actions; test ambiguity, cancellation, retries, duplicate prevention and partial failure.
4. OAuth spike: connect/disconnect, renew credentials, process a sync event, handle revoked access and delete credentials safely.
5. Offline spike: create a task/note offline, restart, reconnect, and verify retries and conflict handling.

10. Decisions still open

- Minimum supported iOS and Android versions.
- Whether full app blocking can be approved and implemented on both platforms.
- Initial AI provider/model and monthly usage budget.
- First Workspace integrations and OAuth verification requirements.
- News licensing/source strategy and content retention policy.
- Privacy policy, user data export/deletion, and retention defaults.

11. Final recommendation

Use Expo + React Native + TypeScript, Supabase/PostgreSQL, focused native Swift/Kotlin modules, SQLite for local reliability, and a server-side provider-agnostic AI Action Engine. This balances development speed and long-term flexibility. Robustness comes from clear boundaries, validation, native-device testing, privacy controls, database migrations and observability—not from adding more technologies early.